SIMATS ENGINEERING

ASSESSMENT TOOL 3 - Output Prediction

COURSE NAME: Data Structures **COURSE CODE:** CSA03

Register Number	**1921260133**
Name	**Abinayasri K**

Assessment Tool Weightage: 35%

COURSE OUTCOME COVERED

CO1: *Apply suitable data structures and ADTs for efficient problem solving by analyzing time and space complexity. (BL3, BL4)*

Questions: 5 **EACH QUESTIONS CARRIES:** 5 MARKS **Total Marks:** 25 **Duration:** ONE HOUR

S. No.	List of Questions	

1	**Array Traversal Program**	`#include <stdio.h>`
		`int main()`
		`{`
		`  int a[5] = {10, 20, 30, 40, 50};`
		`  int i;`
		`  for(i = 0; i < 5; i++)`
		`  printf("%d ", a[i]);`
		`  return 0;`
		`}`

2	**Sum of Array Elements Program**	`#include <stdio.h>`
		`int main()`
		`{`
		`  int a[] = {2, 4, 6, 8};`
		`  int sum = 0, i;`
		
		`  for(i = 0; i < 4; i++)`
		`  sum += a[i];`
		
		`  printf("%d", sum);`
		`  return 0;`
		`}`

3	**Recursive Factorial Program**	`#include <stdio.h>`
		
		`int fact(int n)`
		`{`
		`  if(n == 0)`
		`  return 1;`
		`  return n * fact(n - 1);`
		`}`
		
		`int main()`
		`{`
		`  printf("%d", fact(4));`
		`  return 0;`
		`}`

4	**Recursive Fibonacci Program**	`#include <stdio.h>`
		
		`int fib(int n)`
		`{`
		`  if(n <= 1)`
		`  return n;`
		`  return fib(n-1) + fib(n-2);`
		`}`
		
		`int main()`
		`{`
		`  printf("%d", fib(6));`
		`  return 0;`
		`}`

5	**Function as ADT Operation Program**	`#include <stdio.h>`
		
		`int add(int x, int y)`
		`{`
		`  return x + y;`
		`}`
		
		`int main()`
		`{`
		`  printf("%d", add(15, 25));`
		`  return 0;`
		`}`

Code of Conduct:

I Abinayasri K / 1921260133 *(Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.*
I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:

RUBRICS FOR CALCULATION:

Criteria	Weightage (5 Marks)	Level 4 – Exemplary (5 Marks)	Level 3 – Proficient (4 Marks)	Level 2 – Developing (3 Marks)	Level 1 – Beginning (1–2 Marks)
Understanding of Program Logic	1	Correctly identifies the complete program flow and execution sequence.	Minor misunderstanding of program flow.	Partial understanding of logic.	Unable to interpret the program logic.
Trace of Variables / Function Calls	1	Accurately traces all variable values, iterations, and recursive calls.	Traces most values correctly with minor errors.	Traces some values but misses important steps.	Incorrect or no tracing.
Output Prediction Accuracy	2	Output predicted completely and correctly.	Output mostly correct with minor mistakes.	Output partially correct.	Output incorrect or missing.
Complexity Interpretation (Time/Space)	1	Correctly identifies and justifies complexity where applicable.	Correct complexity with limited justification.	Partial understanding of complexity.	Incorrect or no complexity analysis.

SIMATS ENGINEERING

ASSESSMENT TOOL 3 - Output Prediction
Course: Data Structures (CSA03) | CO1 (BL3, BL4)
Name: Abinayasri K | Register Number: 1921260133

Q	Program	Output
1	Array Traversal	10 20 30 40 50
2	Sum of Array Elements	20
3	Recursive Factorial	24
4	Recursive Fibonacci	8
5	Function as ADT Operation	40

Question 1: Array Traversal

```
#include <stdio.h>
int main()
{
    int a[5] = {10, 20, 30, 40, 50};
    int i;
    for(i = 0; i < 5; i++)
        printf("%d ", a[i]);
    return 0;
}
```

Program logic: The loop visits every index from 0 to 4 and prints each element followed by a space.

Trace:

i	Condition i < 5	a[i]	Printed
0	True	10	10
1	True	20	20
2	True	30	30
3	True	40	40
4	True	50	50
5	False	-	Loop ends

`Output: 10 20 30 40 50`

Complexity: Time = O(n); Space = O(1) extra. The loop runs n times (n = 5) with one print each; only the variable i is used besides the array.

Question 2: Sum of Array Elements

```
#include <stdio.h>
int main()
{
    int a[] = {2, 4, 6, 8};
    int sum = 0, i;

    for(i = 0; i < 4; i++)
        sum += a[i];

    printf("%d", sum);
    return 0;
}
```

Program logic: sum starts at 0 and each iteration adds the current element. The result is printed once after the loop.

Trace:

i	a[i]	sum (before)	sum (after)
0	2	0	2
1	4	2	6
2	6	6	12
3	8	12	20

`Output: 20`

Complexity: Time = O(n); Space = O(1) extra. One pass over n = 4 elements; only sum and i are extra variables.

Question 3: Recursive Factorial

```
#include <stdio.h>
int fact(int n)
{
    if(n == 0)
        return 1;
    return n * fact(n - 1);
}

int main()
{
    printf("%d", fact(4));
    return 0;
}
```

Program logic: fact(n) calls itself with n-1 until the base case n == 0 returns 1. Then results multiply on the way back up.

Trace:

Call	Action	Returns
fact(4)	4 * fact(3)	4 * 6 = 24
fact(3)	3 * fact(2)	3 * 2 = 6
fact(2)	2 * fact(1)	2 * 1 = 2
fact(1)	1 * fact(0)	1 * 1 = 1
fact(0)	Base case	1

`Output: 24`

Complexity: Time = O(n); Space = O(n). There are n + 1 calls (5 here), each doing constant work. The call stack grows to depth n + 1 before unwinding.

Question 4: Recursive Fibonacci

```
#include <stdio.h>
int fib(int n)
{
    if(n <= 1)
        return n;
    return fib(n-1) + fib(n-2);
}

int main()
{
    printf("%d", fib(6));
    return 0;
}
```

Program logic: Base cases: fib(0) = 0 and fib(1) = 1. Every other call returns the sum of the two previous Fibonacci values.

Trace:

n	fib(n)	Computation
0	0	base case
1	1	base case
2	1	fib(1) + fib(0) = 1 + 0
3	2	fib(2) + fib(1) = 1 + 1
4	3	fib(3) + fib(2) = 2 + 1
5	5	fib(4) + fib(3) = 3 + 2
6	8	fib(5) + fib(4) = 5 + 3

`Output: 8`

Complexity: Time = $O(2^n)$ (exponential); Space = O(n). Each call spawns two more, so fib(6) makes 25 calls in total and the same subproblems are recomputed. The maximum recursion depth is n, which gives O(n) stack space.

Question 5: Function as ADT Operation

```
#include <stdio.h>
int add(int x, int y)
{
    return x + y;
}

int main()
{
    printf("%d", add(15, 25));
    return 0;
}
```

Program logic: add() is an operation on the integer ADT: it takes two operands and returns their sum, so main() prints the returned value.

Trace:

Call	x	y	Returns
add(15, 25)	15	25	15 + 25 = 40

`Output: 40`

Complexity: Time = O(1); Space = O(1). A single addition is performed regardless of input values, using no extra memory.